Case Studies

ON

Architecting a Highly Interactive Frontend for a SaaS Invoice System using React.js

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE IN

MASTER OF COMPUTER APPLICATION (MCA)

SUBMITTED TO:

Name: Deepali Saini

Subject:FRONT END TECHNOLOGIES

SUBMITTED BY:

Name:Manish Kumar Mishra

UID: 25MCA20185

Section-MCA-2(A)

UNIVERSITY INSTITUTE OF COMPUTING

CHANDIGARH UNIVERSITY GHARUAN,

MOHALI (PUNJAB)

Title:

Architecting a Highly Interactive Frontend for a SaaS Invoice System using React.js

Problem Statement

Existing invoicing solutions often suffer from poor user experiences characterized by clunky interfaces, frequent page reloads, and a lack of real-time feedback. Users creating complex invoices require instantaneous mathematical calculations for taxes and discounts, seamless navigation, and the ability to preview documents without server delays. There is a critical need for a high-performance, responsive Single Page Application (SPA) that delivers a smooth, desktop-like experience directly within the web browser.

1.Introduction

To solve the UX and performance bottlenecks of traditional web apps, this project was architected as a modern Single Page Application using **React.js**. React was chosen for its component-based architecture, which allows for the creation of reusable, independent UI elements. To ensure lightning-fast development and optimized builds, **Vite** was utilized as the build tool. For styling and accessibility, **Tailwind CSS** combined with **Shadcn UI** was selected to rapidly build a premium, responsive, and highly readable interface without the bloat of traditional CSS files.

2. Objectives

To design and develop a client-side web application that automates the invoicing lifecycle with a focus on seamless user experience. The frontend system will solve the problem statement by:

1. Utilizing a component-driven architecture to ensure modular and maintainable code.
2. Implementing global state management to handle complex, real-time mathematical calculations across the app instantly.
3. Generating and rendering downloadable, high-fidelity PDF documents purely on the client side to eliminate server wait times.
4. Consuming RESTful APIs asynchronously to provide a fluid, non-blocking user experience during authentication and data retrieval.

3. Methodology

The frontend development lifecycle was executed through a structured, modern React ecosystem:

1. **Component-Driven UI Architecture:** The application was broken down into modular React components (e.g., Dashboard, CreateInvoice, DataTables). Routing was handled entirely on the client side using `react-router-dom`, ensuring instant page transitions without browser refreshes.
2. **Advanced State Management:** Managing the state of a complex invoice (with multiple line items, tax rates, and discounts) required more than standard local state. **Zustand** was implemented as a lightweight global state manager. This allowed the application to instantly calculate subtotals and totals in real-time as the user typed, without complex "prop-drilling" between components.
3. **Modern Styling & Accessibility: Tailwind CSS** was used for utility-first styling, ensuring the application is fully responsive across mobile and desktop devices. **Shadcn UI** (built on Radix UI) was integrated to provide accessible, keyboard-navigable components like interactive dropdowns, dialog modals, and form inputs.
4. **Client-Side Document Generation:** A major frontend challenge was rendering PDFs without relying on a backend server. This was solved by integrating the `@react-pdf/renderer` library. It allowed the mapping of React state data directly onto an invisible canvas, generating a downloadable PDF blob securely and instantly within the user's browser.
5. **Data Visualization:** To provide immediate financial insights, the **Recharts** library was used. It takes the raw invoice data fetched from the API and renders an interactive, responsive SVG Area Chart on the dashboard, dynamically updating based on the user's selected currency.
6. **Asynchronous API Integration:** The frontend communicates with the backend REST API using the native `fetch` API combined with React's `useEffect` hooks. JWT tokens are securely stored in `localStorage` and attached to authorization headers, ensuring secure, stateless data fetching.

4.Output Screenshot

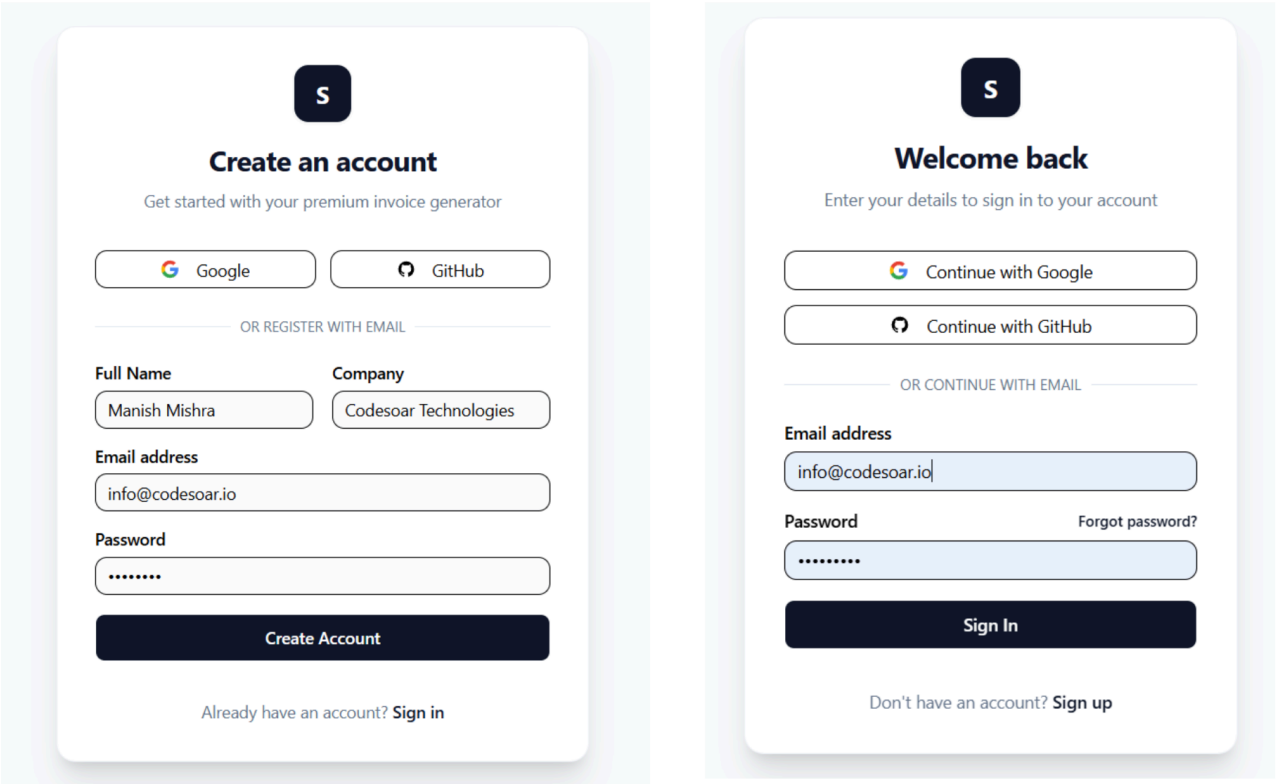

Figure 1
User Authentication Interface. This form captures the user's initial registration details (Name, Email, Password, Company) securely hashing the data before saving it to the database

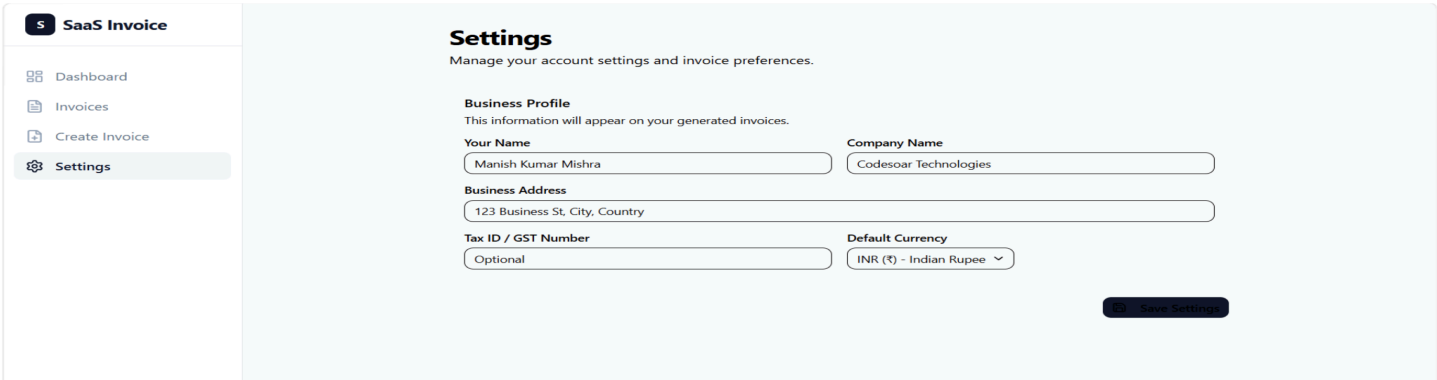

Figure 2
User Profile and Settings. This interface allows users to input their dynamic business details, including GST numbers and preferred default currency (e.g., INR, USD).

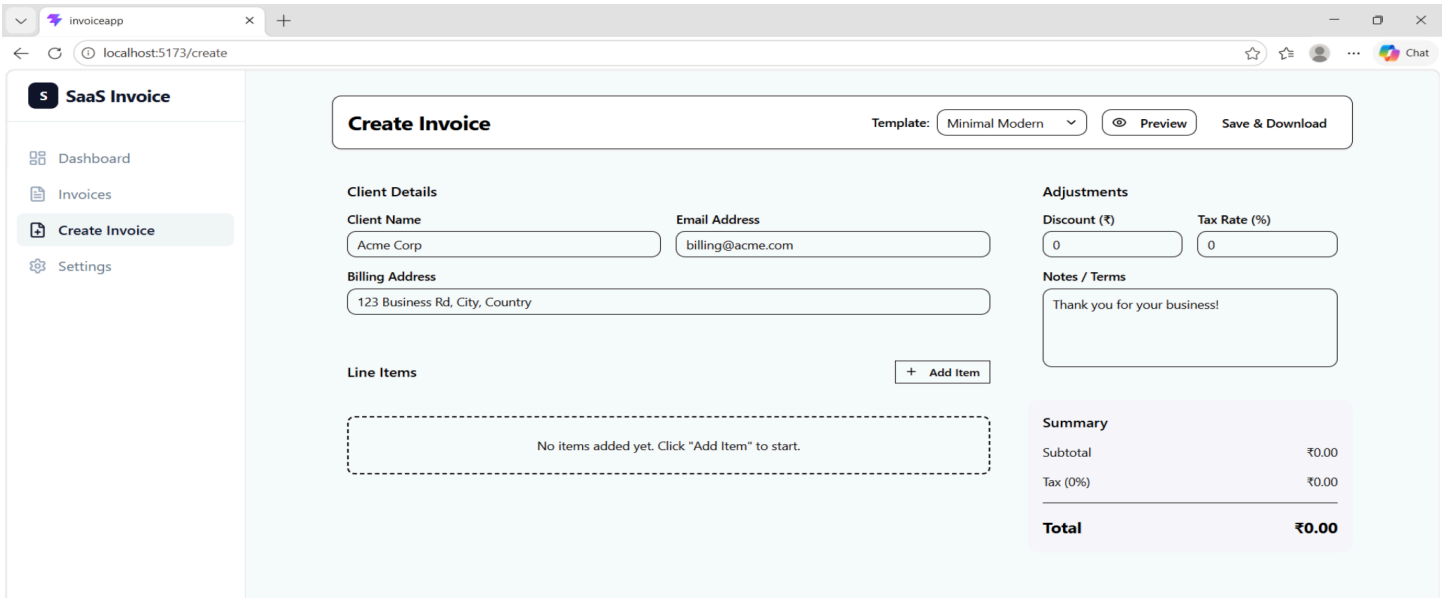

Figure 3
Invoice Generation Form. This input screen collects client details, dynamic line items (Quantity, Price), tax rates, and discount adjustments to calculate the final billing amounts.

Figure 4
Real-time Analytics Dashboard. The output view displaying calculated total revenue, pending amounts, paid invoice counts, and an interactive area chart based on the user's database records.

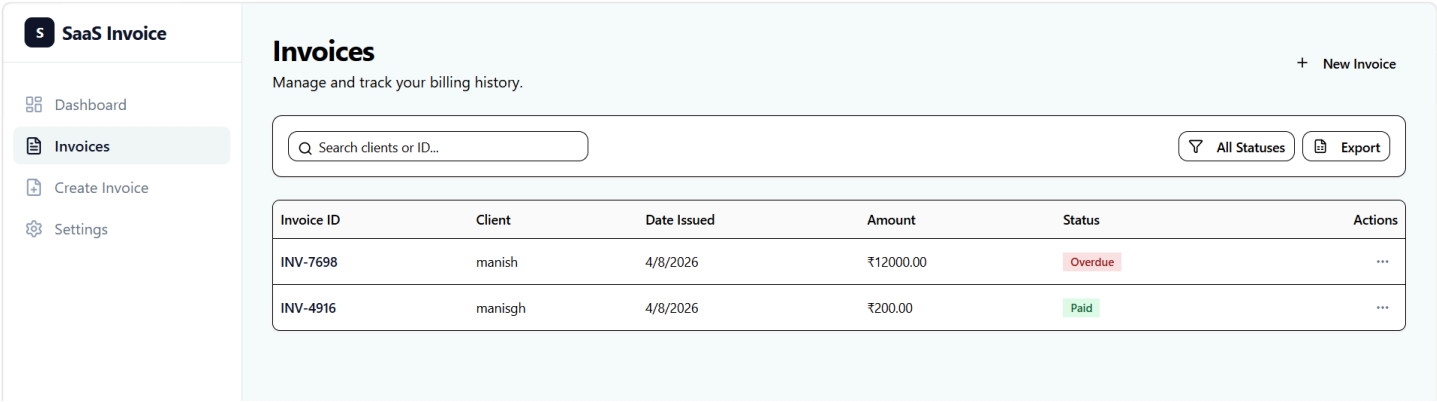

Figure 5
Invoice Management Data Table. The output interface where users can read, filter, update statuses (Paid/Pending), and delete saved invoices.

5. Conclusion

The frontend architecture of the SaaS Invoice Management System successfully demonstrates the power of modern React. By offloading complex tasks—such as mathematical calculations and PDF generation—to the client side, the application drastically reduces server load and delivers an exceptionally fast, app-like user experience. The combination of Tailwind CSS, Zustand, and React created a highly maintainable, scalable, and visually premium frontend product.

6. LEARNING OUTCOME

Working on this frontend-focused case study provided deep practical knowledge in modern UI engineering:

1. **React Hooks Mastery:** Gained extensive experience utilizing `useState`, `useEffect`, and custom hooks to manage component lifecycles and side effects.
2. **Global State Management:** Learned how to architect scalable global state stores using Zustand for complex data structures.
3. **API Consumption:** Mastered asynchronous JavaScript (`async/await`) to connect a React interface to external REST APIs securely.
4. **Responsive UI/UX Design:** Understood the principles of utility-first CSS (Tailwind) to build interfaces that adapt flawlessly to different screen sizes.

7. References

Github:https://github.com/official-Manish82/invoice_app

- **React Documentation:** ***react.dev***
- **Vite Build Tool:** ***vitejs.dev***
- **Zustand State Management:** ***zustand-demo.pmnd.rs***
- **React-PDF Renderer:** ***react-pdf.org***
- **Tailwind CSS Guidelines:** ***tailwindcss.com***
- **Shadcn UI Components:** ***ui.shadcn.com***